Modularity

Kahoot!

Overview

Different platforms offer different mechanisms for grouping code together.

This related code is grouped into modules (although the exact name depends on the platform).

What is a module? Search and define it.

Understanding modularity and its many incarnations is critical for architects.

Modularity is an organizing principle.

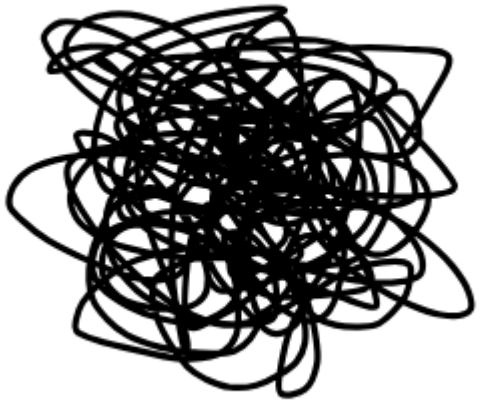
A Physics analogy

Software systems model complex systems that tend toward entropy (what is entropy?).

Energy must be added to a physical system to preserve order.

The same is true for architects.

They need to expend energy to ensure good structural soundness.



Things we do to tame complexity

Learn the domain with Agile planning & discovery practices:

- Event Storming
- User Stories
- Behavior-Driven Development (BDD)

Agile technical practices:

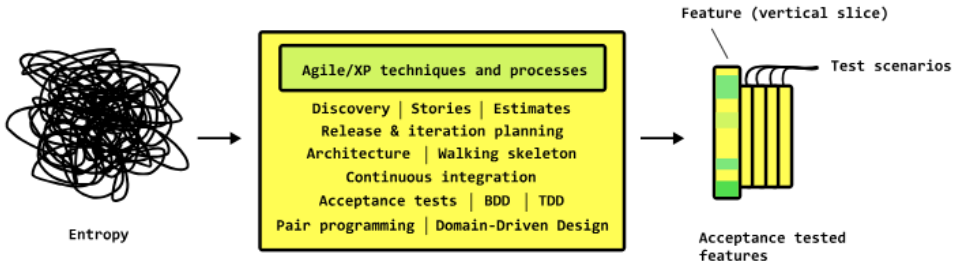
- Domain-Driven Design (DDD)
- Behavior Driven Development (BDD)
- Test-Driven Design (TDD)
- Other: acceptance tests, pair programming to model the domain, catch regressions, and ensure quick feedback loops.

Design patterns and principles:

- SOLID
- YAGNI

Clean code:

- Object Calisthenics
- Conventions
- Simple Design



Modularity

Modularity describes a local grouping of related code.

Most languages provide built-in mechanisms for modularity:

- Functions
- Methods
- Classes
- Packages
- Namespaces

Architects must be aware on how developers package things, since this has implications on the architecture.

Modularity denotes a related grouping of code: classes, functions o any other grouping.

This is a mere logical separation, rather than a physical one.

Measuring Modularity

Giving the importance of this concept, architects need tools to understand it.

These tools could also be used to improve it.

We will cover three concepts:

- Cohesion
- Coupling
- Connascence

Kahoot!

Cohesion

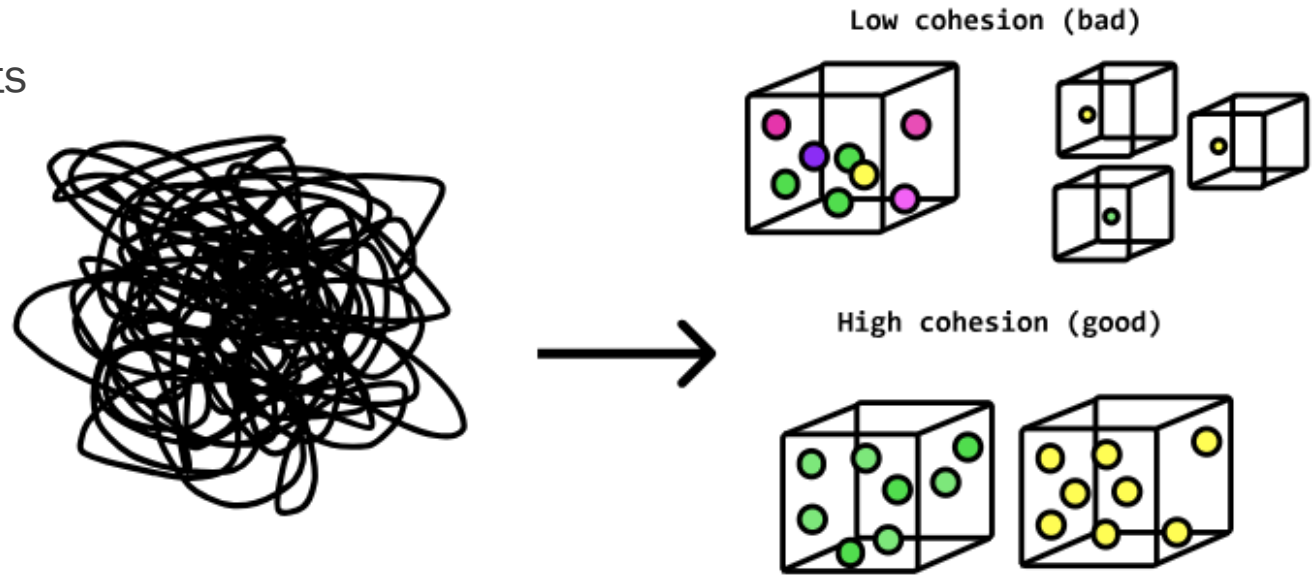
Cohesion refers to what extent the parts of a module belong together.

In other words, is a measure on how the parts of a module are related to each other.

There are several types of Cohesion

1. Functional → [best (more connected)]
2. Sequential
3. Communicational
4. Procedural
5. Temporal
6. Logical
7. Coincidental → [worst (barely connected)]

Let's search the definitions and define them.



Cohesion example

```
// Bad cohesion - not focused on a single responsibility

class RequestHelpers {
  ...
  createUser (details) { ... }
  getEmailFromRequest (req) { ... }
  checkForumExists (req) { ... }
  isAuthenticated (req) { ... }
  endTransaction (req) { ... }
}

// Good cohesion

class TextUtil {
  public static strip (text) { ... }
  public static isPalindrome (text) { ... }
  public static replaceAll (text, replacement) { ... }
  public static toCamelCase (text) { ... }
}
```

Example

Customer Maintenance	Customer Maintenance
Add customer	Add customer
Update customer	Update customer
Get customer	Get customer
Notify Customer	Notify Customer
Get Customer Orders	
Cancel Customer Orders	**Order Maintenance**
	Get Customer Orders
	Cancel Customer Orders

Which one is better?

Direct Object

Cohesion

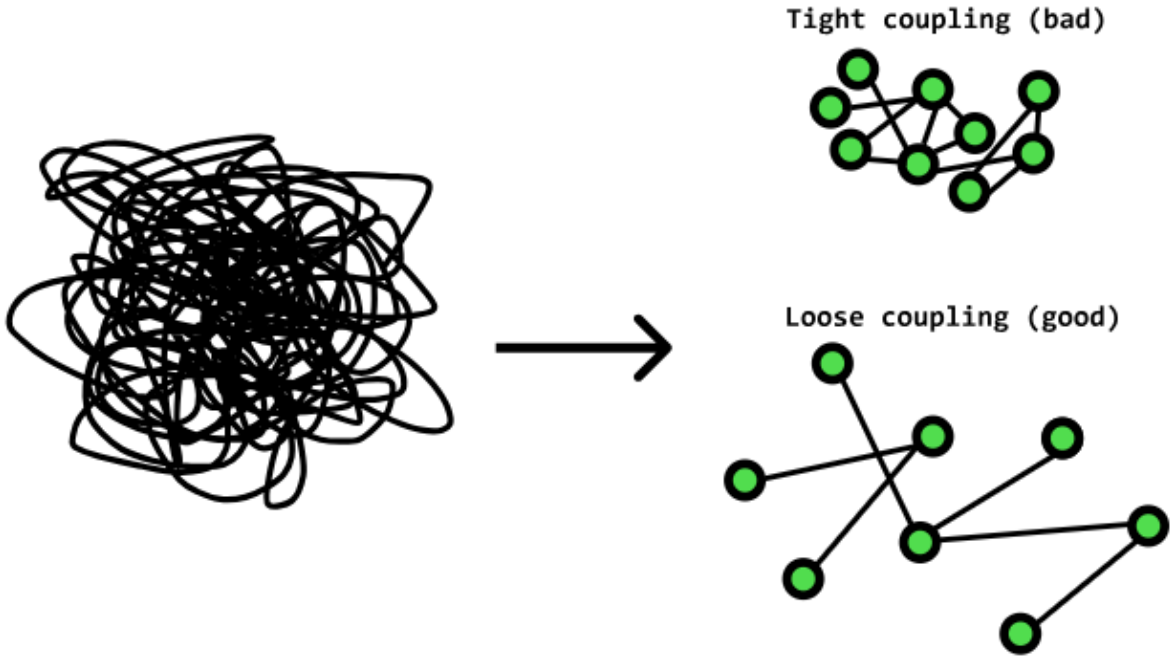
Type	Description
Functional	**The best and most connected form.** All elements of a module contribute to a single, well-defined task.
Sequential	The **output from one part serves as the input to another** part.
Communicational	They all **operate on the same data** or information.
Procedural	They always **follow a specific sequence of execution**.
Temporal	They are all **processed at the same time** (e.g., all initialization steps).
Logical	They are **logically similar** even if they are different in nature (e.g., grouping all input/output routines).
Coincidental	**The worst form of cohesion.** The elements have **no meaningful relationship** and are bundled together by chance.

Coupling

Coupling measures two things: incoming and outgoing connections to a coding artifact.

These measures are called afferent and efferent coupling.

Investigate why are they called this way.



Afferent and Efferent Coupling

Afferent Coupling (C_a)

- measures **who depends on you**. It counts the number of classes outside a specific package that depend on classes inside that package.

Efferent Coupling (C_e)

- measures **who you depend on**. It counts the number of classes inside your package that depend on classes outside your package

The "Instability" Formula

- Architects use these two numbers to calculate the Instability **I** of a component. This helps determine if a design is rigid or flexible.

$$I = \frac{C_e}{C_a + C_e}$$

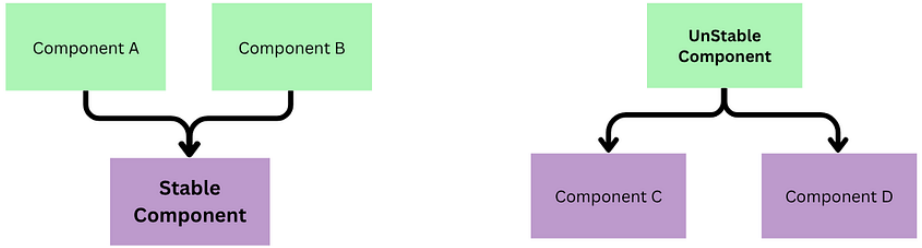
I = 0: The component is maximally stable (lots of people depend on it, but it depends on no one). Think of a core library like String or Math.If

I = 1: The component is maximally unstable (it depends on many things, but no one depends on it). This is often where "top-level" application logic sits.

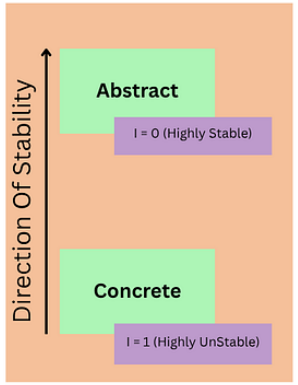
Instability

	Foundation	Orchestrators	Plugins
Zone	Stable Zone	Middle Zone	Unstable Zone
Range	$I \in [\ 0\ ,\ 0.2\ ]$	$I \in [\ 0.3\ ,\ 0.6\ ]$	$I \in [\ 0.7\ ,\ 1.0\ ]$
What they are	• Low-level libraries • Core business entities • "leaf" nodes (e.g., a Money class)	• Service layers • Coordinators between foundation and outside world.	• User Interface (UI) code • Glue code • Main entry points.
Why	• Depended on (high C_a) • Few dependencies (low C_e)		• Lot of dependencies (high C_e) • Not depended on (zero C_a)
Norm	• These should be very hard to change • They should be highly tested and abstract	• Normal for to have a mix • "meat" of application logic.	• We *want* our UI to be easy to change without affecting the core logic.
Types	• Interfaces/Abstract Classes	• Business Logic	• UI / Entry Points / Config
Flexibility	• Low (Should be rigid/stable)	• Moderate	• High (Should be easy to swap)

Instability and Abstractness



Fan In : Number of components that depend on this component.
Fan Out: Number of components this component depends on.



$$\text{Instability (I)} = \frac{\text{Fan Out}}{\text{(Fan In + Fan Out)}}$$

Range of I can be [0,1]

- **I = 0 →** *Highly stable component (lots of incoming, no outgoing dependencies)*
- **I = 1 →** *Highly unstable (lots of outgoing, no incoming)*

Examples:
- *Fan-out > 0, Fan-in = 0 → Unstable and dependent*
- *Fan-out = 0, Fan-in > 0 → Stable and independent*

Stable Dependencies Principle

- Depend in the direction of stability.

Stable Abstractions Principle

- A component should be as abstract as it is stable.
  - **Stable components** (used by many others) should be abstract, so they can be extended without needing modification.
  - **Unstable components** (likely to change) should be concrete, so changes are easy to make.

Abstraction

- $A = \frac{N_a}{N_c}$,
  - where N_a are abstract classes and interfaces and N_c are the total number of classes in a component
  - **A = 0** the component is fully concrete
  - A = 1 the component is Fully abstract

Main Sequence Graph

The Zone of Pain:

- Components that are highly stable (I is low) but also very concrete (not abstract). Hard to change because of dependencies.

The Zone of Uselessness:

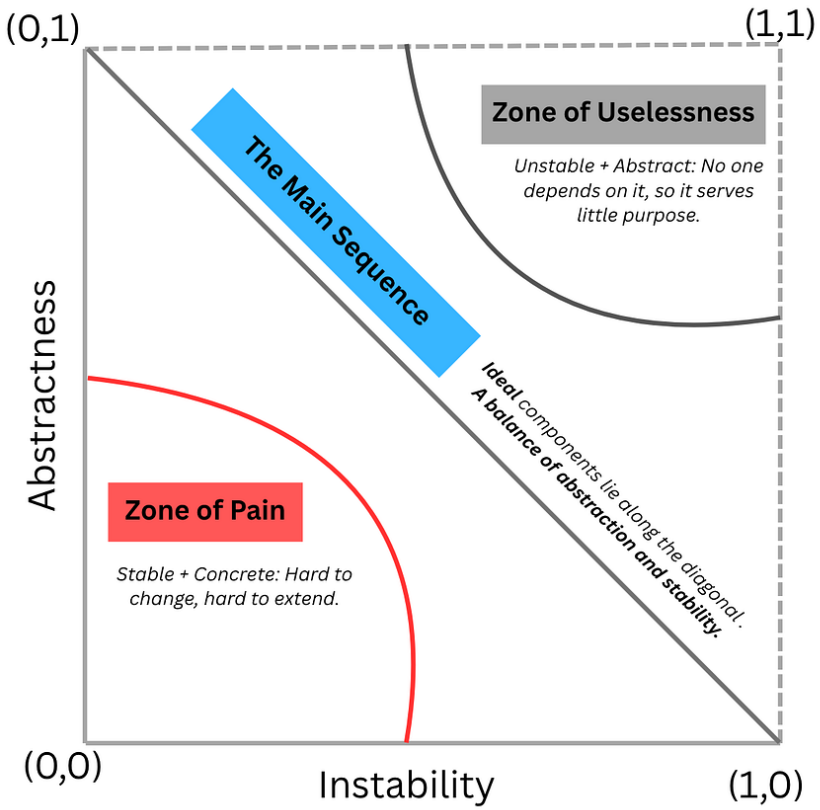
- Components that are highly unstable (I is high) but also highly abstract. No one uses them, and they don't do anything concrete.

The Main Sequence:

- The "healthy" diagonal line where stability and abstractness are balanced.

Distance from Main Sequence

- $D = |A + I - 1|$
  - D=0 Perfectly balanced component
  - D=1 As far from ideal as it gets



Red Flags

The Inversion:

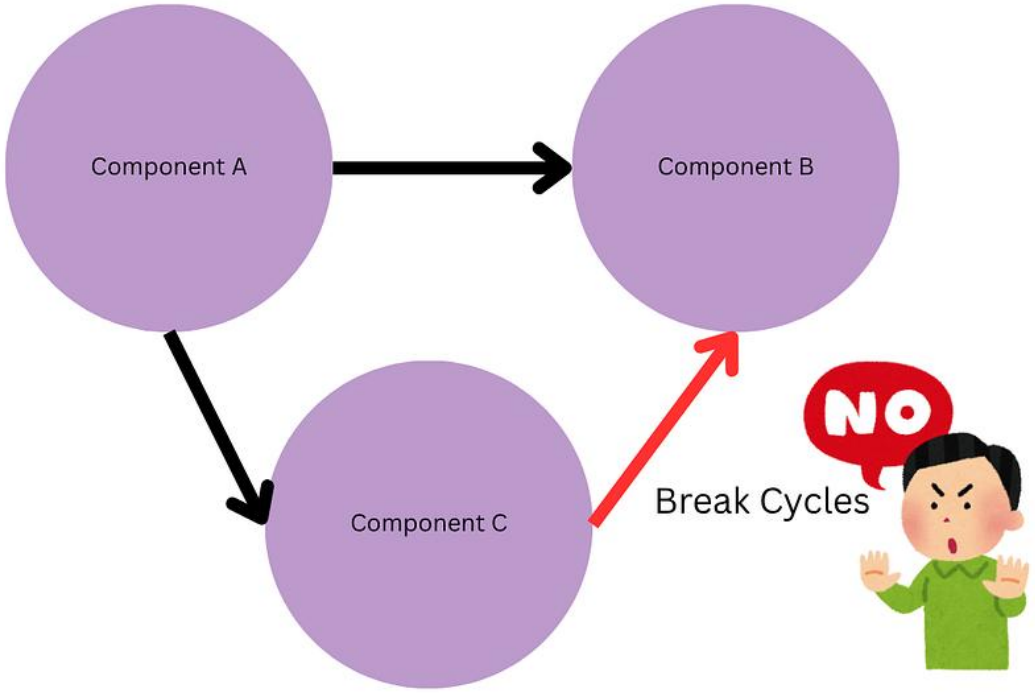
- A core business rule class has an I=0.9. This means your "heart" of the system depends on volatile things like a specific database or a third-party API.

High C_a + High C_e:

- If a class has many incoming and many outgoing dependencies, it’s a "God Object." It’s a bottleneck that is both hard to change and likely to break frequently.

Circular Dependencies:

- If Component A depends on B, and B depends on A, the math breaks down and you end up with a "Big Ball of Mud" where nothing is truly stable.

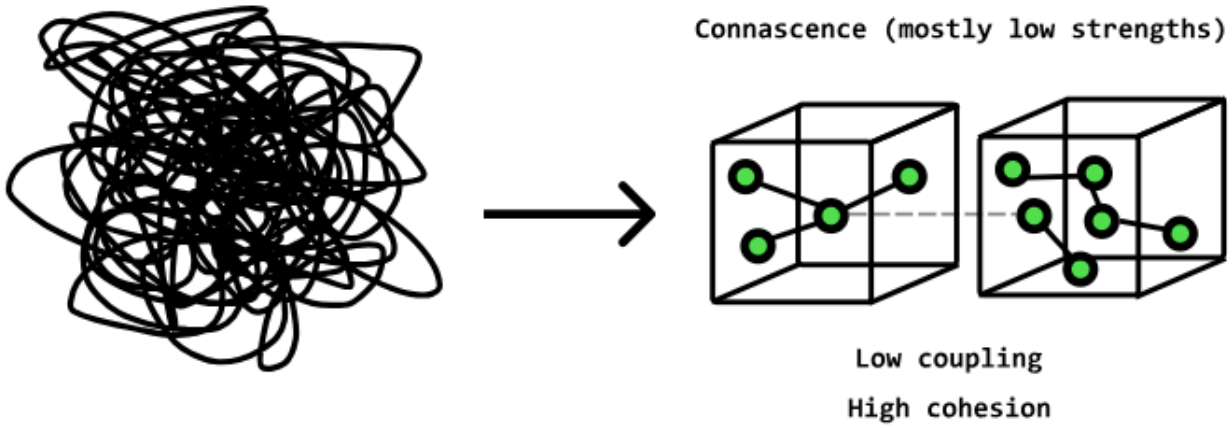


Kahoot!

Connascence

Term originates in 1996 by Meilir Page-Jones.

Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system.



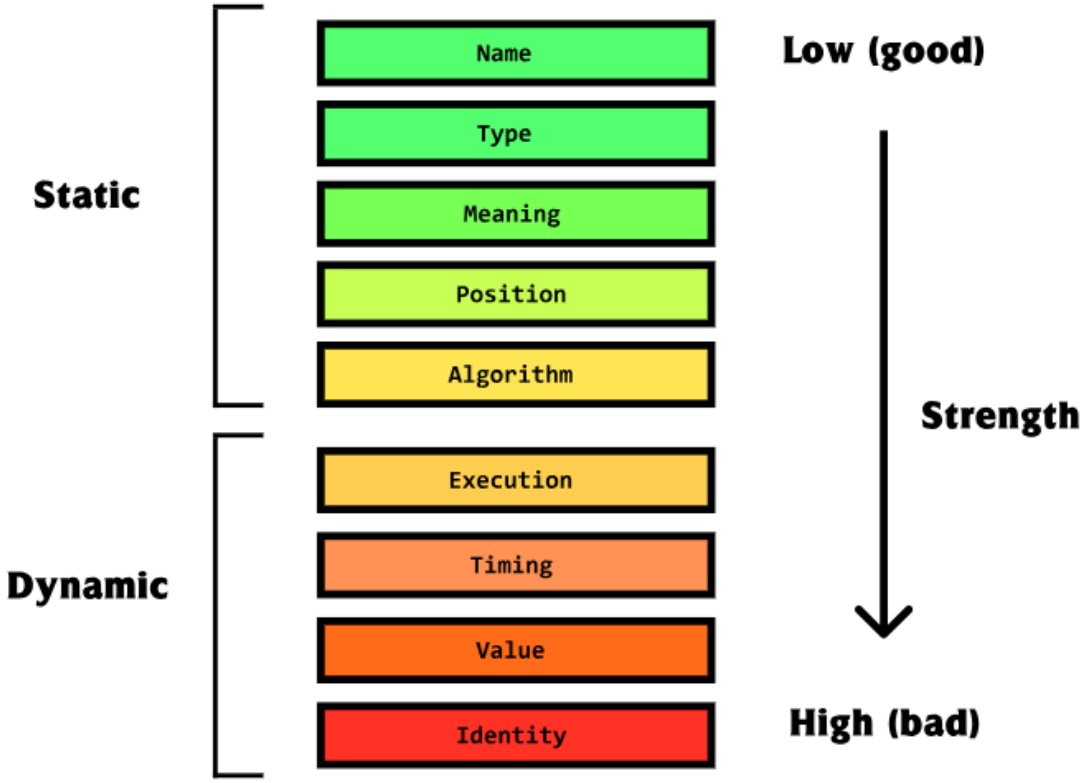
Connascence

He developed two types of connascence: static and dynamic.

- The static refers to the source-code level coupling.
- Whereas the dynamic is used for execution-time coupling.

Activity:

- Investigate the types of static and dynamic connascence and how to transform the connascence of meaning into connascence of name and why is the latter better?



Static connascence

Type	Brief Description	Example	The Fix
Name	Multiple parts must agree on the name of an entity.	Renaming a method calc() to calculate() breaks callers.	Use automated refactoring tools (IDE) to update all references.
Type	Multiple parts must agree on the data type of an entity.	Changing an int age to a string "25" breaks logic.	Use strong typing or interfaces to enforce the contract.
Meaning	Multiple parts must agree on what a specific value represents.	status = 1 (where 1 means "Active" and 2 means "Closed").	Replace magic numbers/strings with **Enums** or **Constants**.
Position	Multiple parts must agree on the order of values.	sendEmail(to, subject, body) – swapping 'to' and 'subject'.	Use **Named Arguments** or pass a **Data Object (DTO)**.
Algorithm	Multiple parts must agree on a specific calculation/logic.	A checksum must be calculated the exact same way on both ends.	Extract the logic into a **Single Shared Utility** or library.

Refactoring Example

FROM POSITION

```
// The Function
function printAddress(addressArray) {
    console.log(`Shipping to ${addressArray[0]}, ${addressArray[1]} ${addressArr
}

// The Call (Highly fragile: What if I swap City and Street?)
printAddress(["123 Maple St", "New York", "10001"]);
```

TO NAME

```
// The Function
function printAddress({ street, city, zip }) {
    console.log(`Shipping to ${street}, ${city} ${zip}`);
}

// The Call (Safe: The order of properties doesn't matter)
printAddress({
    zip: "10001",
    street: "123 Maple St",
    city: "New York"
});
```

Dynamic Connascence

Type	Brief Description	Example	The Fix
Execution	The order of calls matters for the system to work.	Must call open() before read() or the app crashes.	Use the **State Pattern** or **Fluent API** to guide the caller.
Timing	The exact time of execution affects the outcome.	Two threads updating a bank balance at the exact same millisecond.	Use **Locks**, **Semaphores**, or **Atomic Operations**.
Values	Several values are related and must change together.	A "Start Date" must always be logically before an "End Date."	Encapsulate the values into a **Value Object** with validation.
Identity	Multiple parts must reference the exact same object instance.	Two different services sharing one "Live Configuration" object.	Favor **Immutability** or pass specific data instead of the whole object.

Refactoring Execution: Fluent API

WORDY

```
// Is 50 the height or the width? Is 'true' for 'bold' or 'italic'?
const button = new Button("Submit", 50, 100, true, false);
```

FLUENT API

```
const button = new ButtonBuilder()
    .withText("Submit")
    .withWidth(50)
    .withHeight(100)
    .asBold()
    .build();
```

Refactoring Execution: State Pattern

FRAGMENTED LOGIC

```
if (order.isPaid && !order.isShipped) {
    order.ship();
} else {
    throw new Error("Cannot ship!");
}
```

STATE OBJECT

```
// Usage Example
const myOrder = new Order();
myOrder.ship(); // Error: Cannot ship an unpaid order.
myOrder.pay();  // Transitioning to PaidState
myOrder.ship(); // Transitioning to ShippedState
```

Refactoring Execution: State Pattern Implementation

```javascript
// 1. The Context Class: Maintains the current state
class Order {
    constructor() {
        this.state = new DraftState(this); // Initial state
    }

    setState(state) {
        console.log(`Transitioning to ${state.constructor.name}`);
        this.state = state;
    }

    pay() {
        this.state.pay();
    }

    ship() {
        this.state.ship();
    }
}

// 2. Concrete State: Draft
class DraftState {
    constructor(order) {
        this.order = order;
    }

    pay() {
        console.log("Payment processed successfully.");
        this.order.setState(new PaidState(this.order));
    }

    ship() {
        console.log("Error: Cannot ship an unpaid order.");
    }
}

// 3. Concrete State: Paid
class PaidState {
    constructor(order) {
        this.order = order;
    }

    pay() {
        console.log("Error: Order is already paid.");
    }

    ship() {
        console.log("Order is being prepared for shipping...");
        this.order.setState(new ShippedState(this.order));
    }
}

// 4. Concrete State: Shipped
class ShippedState {
    constructor(order) {
        this.order = order;
    }

    pay() {
        console.log("Error: Cannot pay for an order already shipped.");
    }

    ship() {
        console.log("Error: Order is already shipped.");
    }
}
```

Connascence properties

Connascence is a metric, however qualitative, rather than quantitative

Takes a more holistic approach considering three separate axes:

- **Strength**: measures how easy it is for a developer to refactor some types of connascence.
- **Degree**: this relates to the size of the impact – does it impact some classes or many?
- **Locality**: this measures the proximity of the modules to each other. Proximal code often has higher and better types of connascence.

Calculation could be though in terms of cost:

- $Connascence = Strength \times Degree \times \frac{1}{Locality}$

	Low	High
Strength (Type)	Name, Type (Static)	Timing, Identity (Dynamic)
Degree (Scale)	Changing a method name affects 2 files	Changing a global configuration object affects 500 files
Locality (Distance)	Two separate Microservices (I is high)	Two methods in the same Class (I is low).

Using the “formula” in practice

Scenario	Scenario Description	Strength	Degree	Locality	Verdict
Private Method Name	Change the private method fn_123() to calculateValues()	Low (Name)	Low (1-2 calls)	High (Same Class)	**Ignore.** Low cost.
Database Schema	If you change a column in your database from is_active (a boolean) to status_level (an integer)	Med (Meaning)	High (Whole App)	Low (Cross-layer)	**Danger.** Needs a DTO/Interface
Distributed Race Condition	Service A sends a message to Service B, but Service B assumes Service C has already updated the record. If the network is slow and Service C is late, the data gets corrupted	High (Timing)	Med (Several Services)	Very Low (Network)	**Critical.** Refactor immediately.

$$Cost = Strength \times Degree \times \frac{1}{Locality}$$

Use of connascence

There are three guidelines to use the connascence to improve the system modularity:

- Minimize overall connascence by breaking the system into encapsulated elements.
- Minimize any remaining connascence that crosses encapsulation boundaries.
- Maximize the connascence within encapsulated boundaries.

Also, there are good advices from Jim Weirich:

- Rule of degree: Convert strong forms of connascence into weaker ones.
- Rule of locality: As the distance between software elements increases, use weaker forms of connascence.

If you have...	The Strategy is...
High Strength + Low Locality	**Decouple:** Use an abstraction (API/Interface) to turn it into Connascence of Name.
High Degree + High Strength	**Encapsulate:** Hide the complexity inside a single module so the "Degree" drops to 1.
Low Strength + High Locality	**Accept:** This is "Clean Code." Don't over-engineer it.

Component-based thinking

FOUNDATIONS

Kahoot!

Overview

Architects usually think in terms of components (physical manifestation of a module).

Components:

- offer a language-specific mechanism to group artifacts together.
- the simplest component wraps code a higher level of modularity than classes.
- we call this a library.
- also appear as subsystems or layers of an architecture.

A service tends to run its own address space and communicates via low-level network protocols. These components form a standalone, deployable unit.

Components form the fundamental building block in architecture. They are critical consideration for architects.

Architect role

The component …

- is the lowest level of the software system the architects interacts directly with.
- consist of classes or functions, whose design falls under the responsibility of tech leads or developers.

The architect ….

- defines, refines, manages and governs components within an architecture.
- collaborate with business analysts, subject matter experts, developers, QA engineers, operations and enterprise architects.
- should avoid micromanaging each decision from top to bottom.
- must identify components as one of the first tasks on a new project.

Architecture Partitioning

Components represent a general containership mechanism.

- Therefore, architects can build any type of portioning they want.

Several types exist already and will be discussed as part of the architecture styles.

Here we discuss the top-level portioning in an architecture.

- Example: layered monolith vs modular monolith.

The top-level portioning is of a particular interest for architects

- it defines the fundamental architecture style and way of portioning code.

Usually there are two types of top-level portioning:

- technical-based or
- domain- based.

Domain Partitioning

This is inspired by the domain-driven-design (DDD).

In DDD

- The architect identifies domains or workflows interdependent and decoupled from each other.
- For instance, in the microservices architecture.

In a monolith approach

- The architects partitions around workflows rather than technical capabilities.

Technical Partitioning

Architects using technical partitioning organize the components of the system by technical capabilities:

- Presentation.
- Business.
- Rules.
- Persistence.

One of the organizing principles is the separation of technical concerns.

- This creates useful levels of decoupling:
- if a given layer is only connected to some others, this reduces the impacts of changes among them.

Most realistic systems require workflows that cut across technical capabilities.

Neither of these styles is more correct than the other.

Example: Silicon Sandwiches

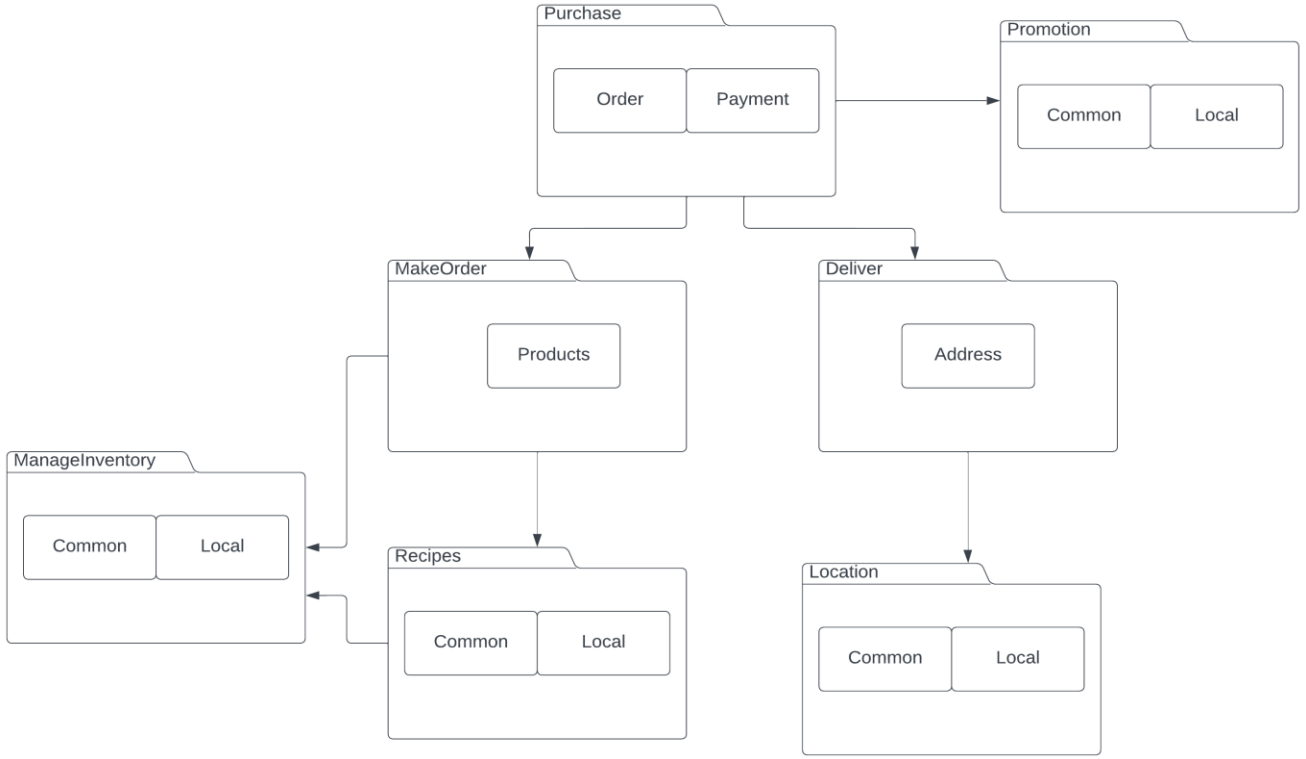
DOMAIN PARTITION:

- Purchase.
- Promotion.
- MakeOrder.
- ManageInventory.
- Recipes.
- Delivery.
- Location.

TECHNICAL PARTITION:

- Common.
- Local.
- Purchase.
- Deliver.

Example: Domain Partitioning



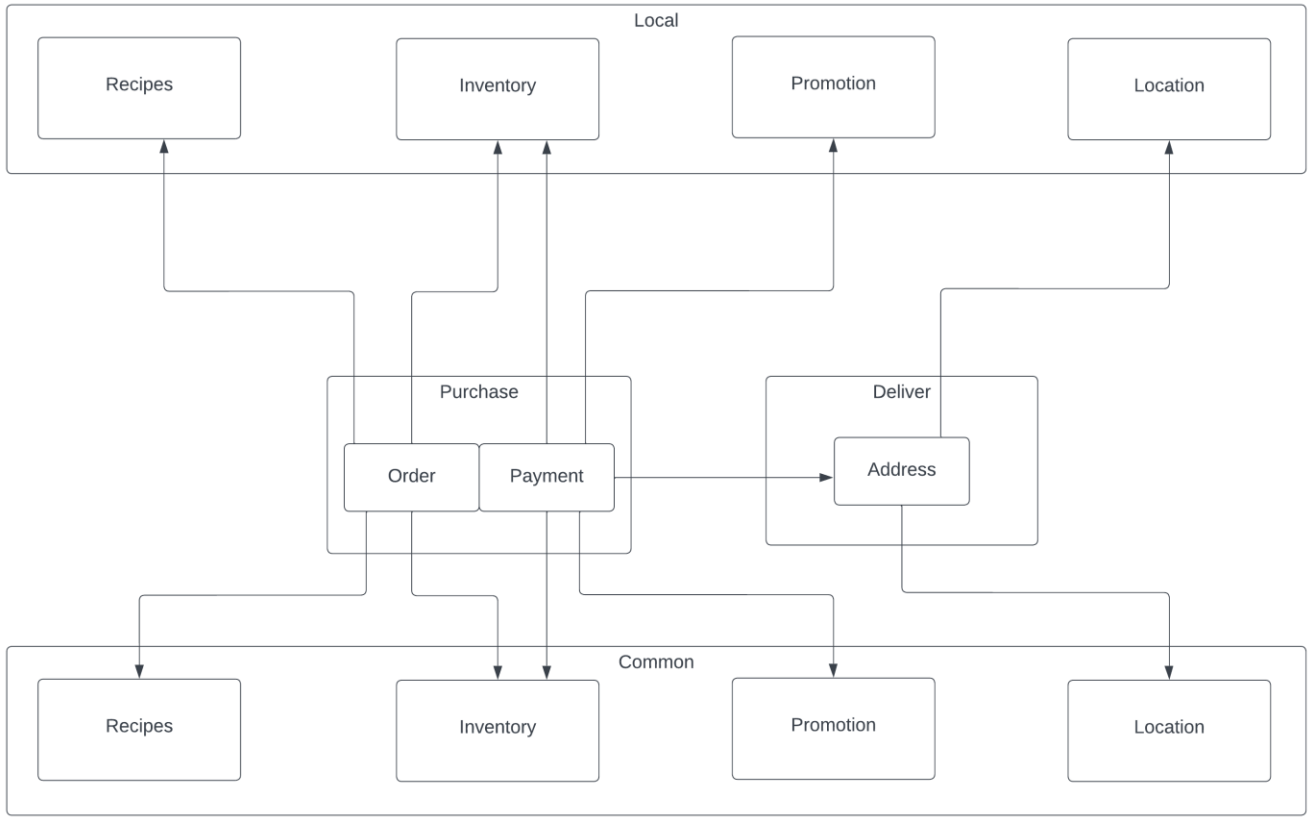
Advantages:

- Modeled more closely toward how business functions.
- Aligns more closely to modular monolith and microservices architecture.
- Message flow matches problem domain.
- Easy to migrate data and components to distributed architecture.

Disadvantages:

- Customization code appears in multiple places.

Example: Technical Partitioning



Advantages:

- Clearly separates customization code.
- Aligns more closely to the layered architecture pattern.

Disadvantages:

- Higher degree of global coupling.
- Developers may need to duplicate domain concepts in both the Common and Local layers.

Developer Role

Developers typically take components, jointly designed with the architecture role.

They further subdivide them into classes, functions or subcomponents.

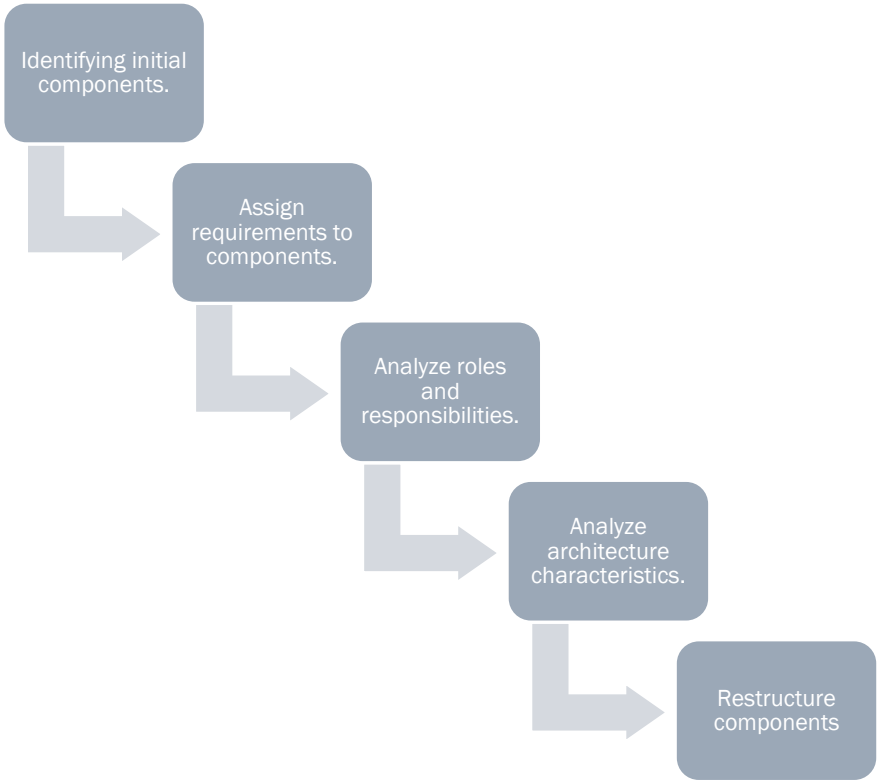
Developers should never take components designed by architects as the last word.

Component Identification Flow

Component identification works best as an iterative process, producing candidates and refinements though feedback.

The cycle has 5 steps:

- Identifying initial components.
- Assign requirements to components.
- Analyze roles and responsibilities.
- Analyze architecture characteristics.
- Restructure components.



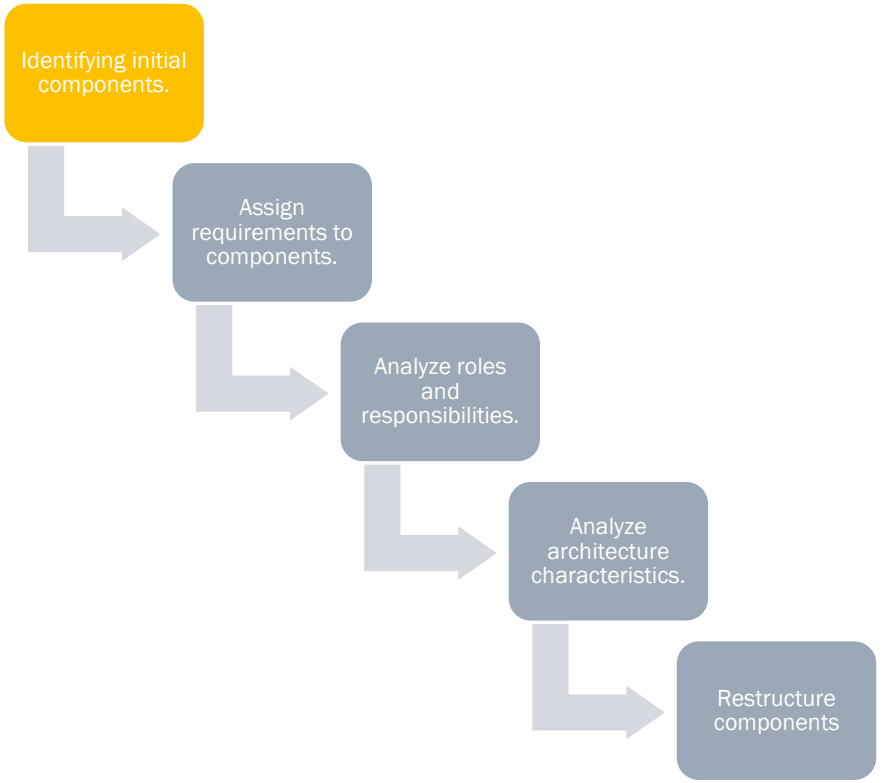
Identifying initial components.

Before any code exists, an architect must determine what top-level components should be there at the beginning.

An architect has the freedom to make up whatever components they want, then map domain functionality to see where the behavior should reside.

The likelihood of achieving a good design from the start is very small.

This is why architects should iterate on component design to improve it.



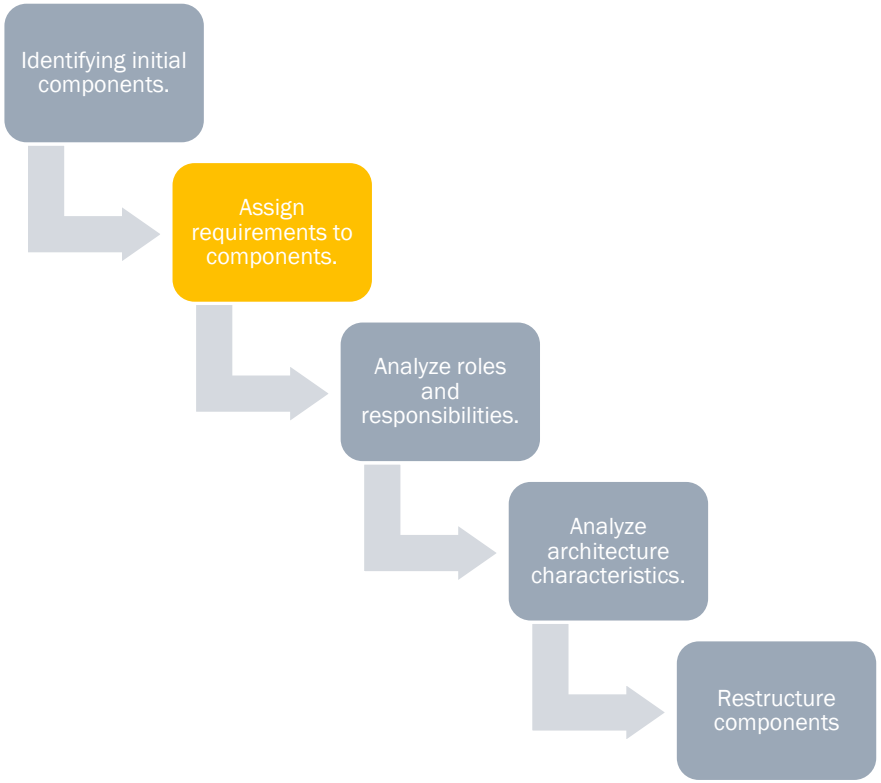
Assign requirements to components

Once the architect has the initial components, they should align this to requirements to see how the fit.

This could create entirely new components, consolidate existing ones, or breaking components apart.

This mapping doesn't have to be exact.

Now the cycle begins between the three remaining steps.



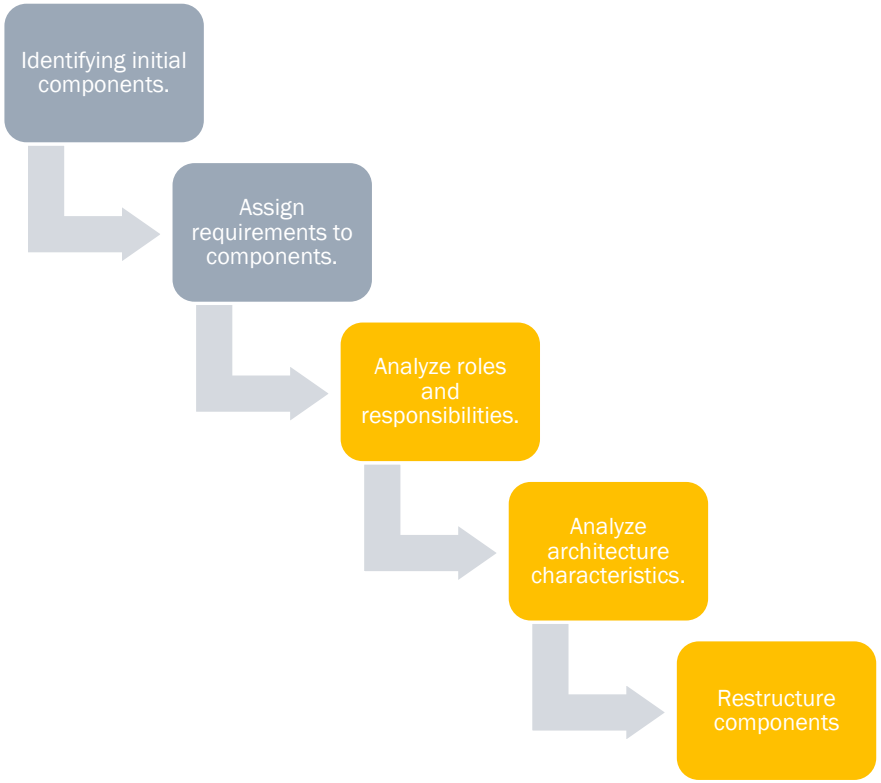
Component Identification Cycle

First the architect analyzes the roles from the requirements and match them to components to define the granularity.

- Then, they should look at the architecture characteristics and think about how they impact component division and granularity.

After, feedback comes in.

- Feedback is important since designing software provides all kinds of unexpected difficulties.
- As architectures and developers delve more into building the application, they gain a more nuanced understanding of the behavior and the roles, and where these should be.



Discovering Components

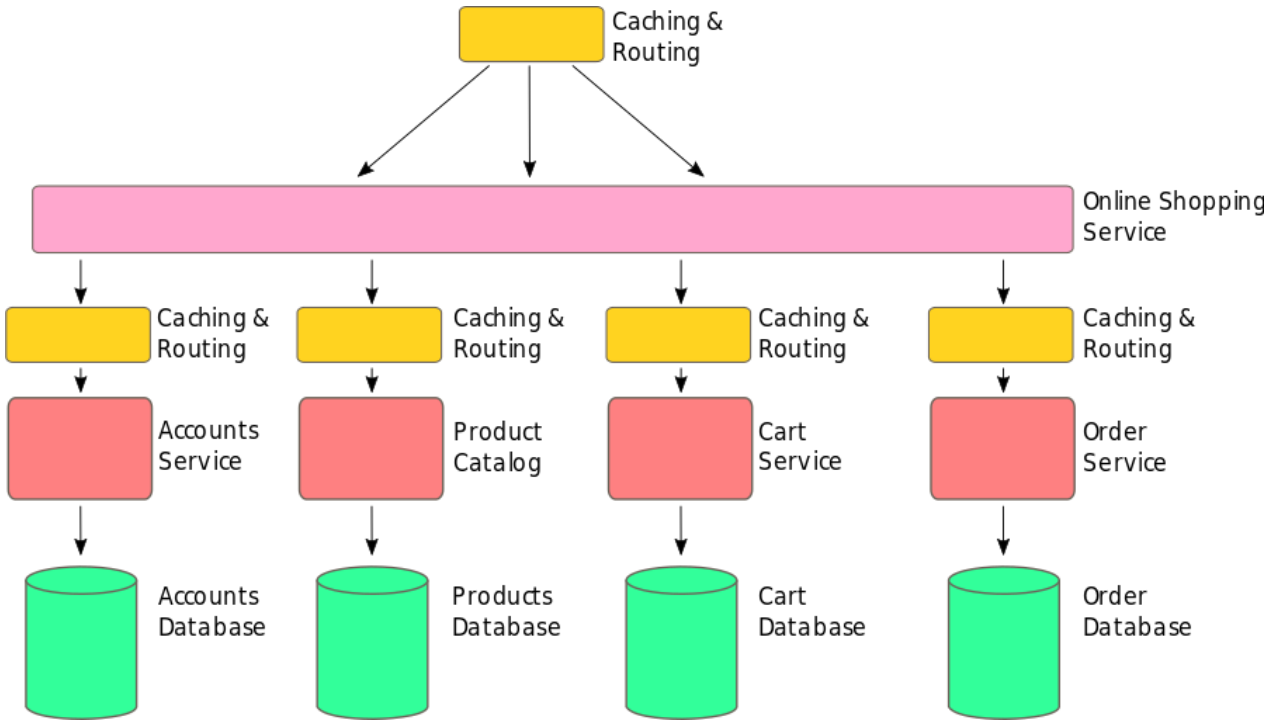
Architects often collaborate with other roles to design components.

- These include business analysts, developers and subject matter experts.

The group also chooses on how to decompose the initial design based on technical or domain partitioning.

- We must be careful of the entity-trap anti-pattern. Search on the web what this is.

We will cover some ways architects can use to discover components.



Actions/Actors Approach

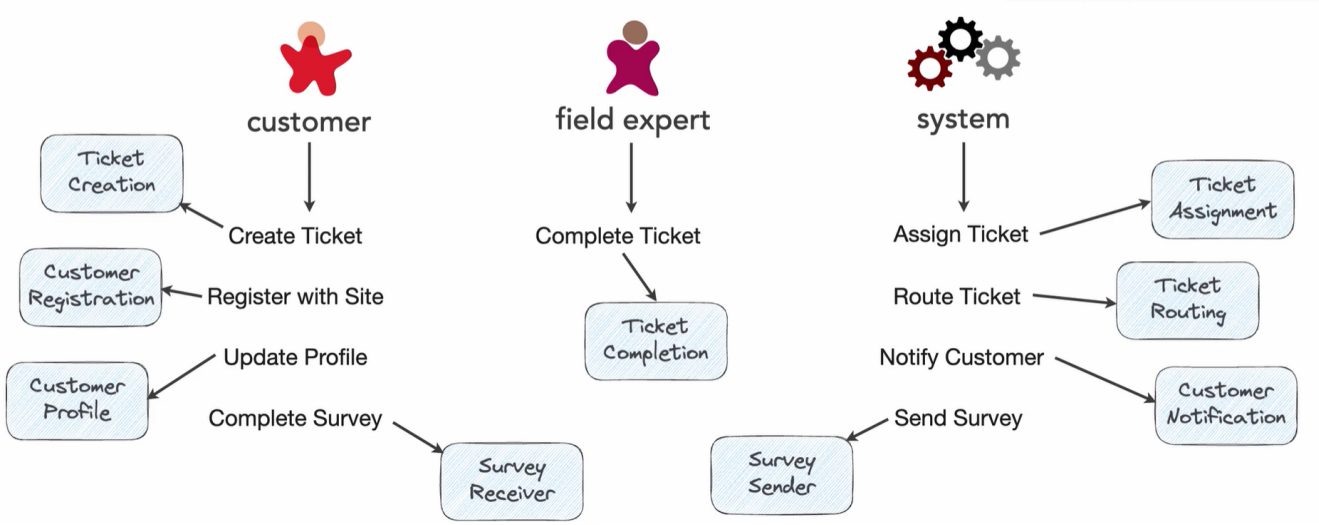
This is a popular way for architects to map requirements to components.

Architects identify **actors** who perform activities with the application.

- These **actors** may perform certain **actions** within the application.

This mapping leads to an initial idea on what components are involved in the different workflows of the application.

This is still a popular idea and works well with both monolithic or distributed architectures.



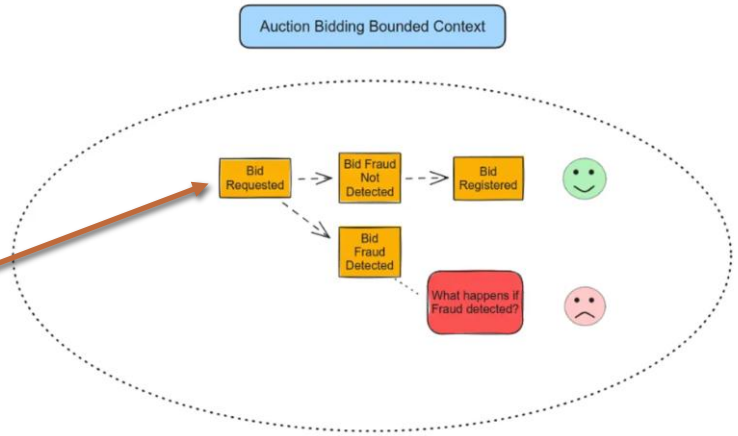
Event Storming

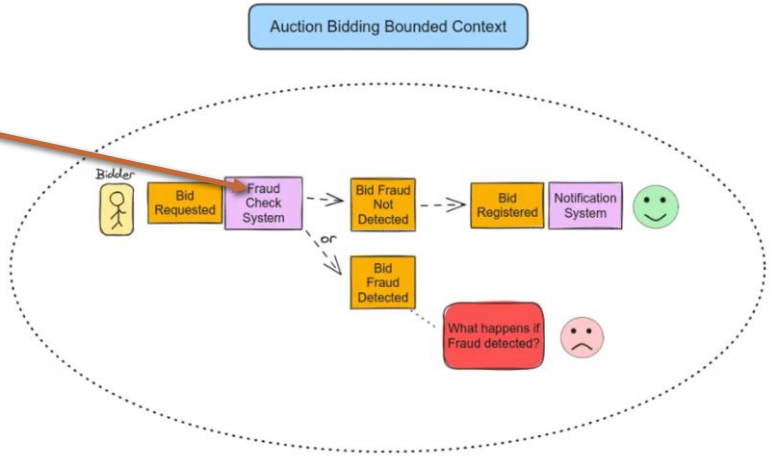
This is component discovery technique, and it comes from the domain-driven- design approach.

The architect assumes the project will use **messages and/or events** to communicate between the different components.

- With this idea, identifies where this messages/events occur, and the actors involved.
- Then creates components around these actors.

This technique is popular with microservices architecture or other distributed architectures where events/messages may happen.





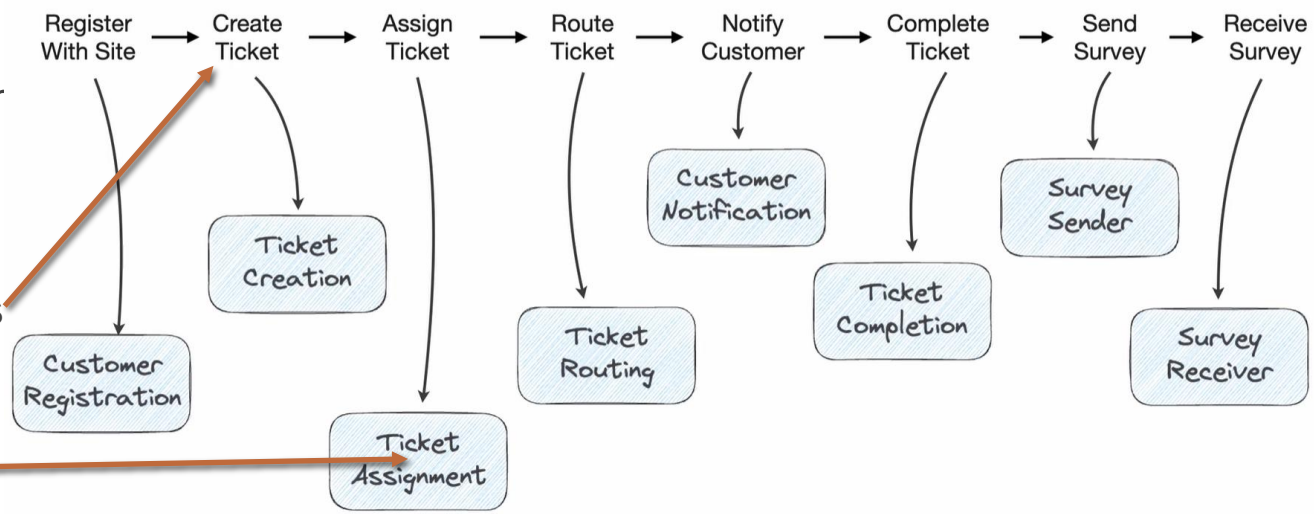
Workflow Approach

An alternative to event storming not relying on messages or DDD.

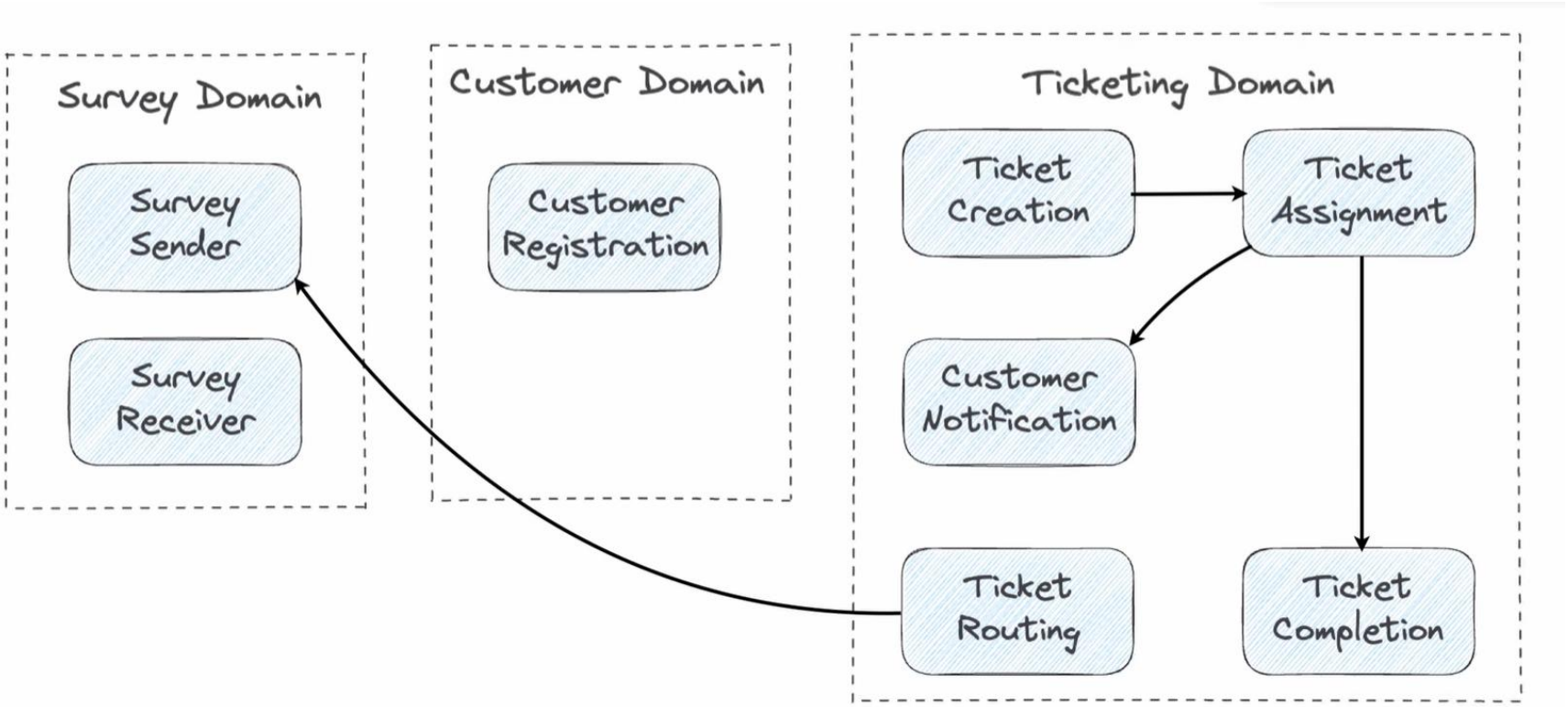
Here the components are modeled based or workflows.

- It identifies the key roles
- Determines the kinds of workflows these roles engage in

Builds components around the identified activities.



Component identification



Quizlet